# File Integrity Monitoring

## Real-Time vs. Polling Detection Latency

**Author:**

**DAANIYA ABSAR**

*Security Lab Report & Technical Analysis*

November 5, 2026

# Contents

**1 Introduction** **2**

**2 Experimental Setup** **2**

**3 Methodology and Execution** **2**
3.1 Step 1: Modifying the Files . . . . . 3
3.2 Step 2: Verifying the File Content . . . . . 3

**4 Results and Analysis** **3**
4.1 Alert Data . . . . . 4
4.2 Calculating Detection Latency . . . . . 4
4.3 Visualizing the Metrics . . . . . 5

**5 Security Implications** **5**

**6 Conclusion** **6**

# 1 Introduction

File Integrity Monitoring (FIM) is a critical security control used to detect unauthorized changes to files and directories. These changes could indicate a security breach, malware infection, or configuration drift. FIM is a core component of compliance standards such as PCI DSS (Requirement 11.5) and HIPAA.

As shown in the Wazuh interface (Figure 1), FIM alerts are generated for file changes, including permissions, content, ownership, and attributes.

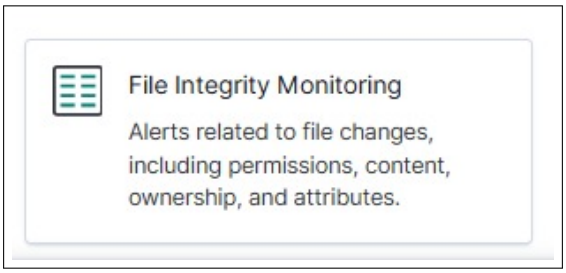



**Figure 1:** File Integrity Monitoring overview card in Wazuh.

There are two primary ways FIM engines watch for changes:

- **Real-Time Monitoring:** Uses OS-level kernel hooks (like `inotify` on Linux) to watch file descriptors. It triggers an alert immediately upon a write, delete, or permission change.
- **Scheduled Polling:** The FIM engine runs periodic directory scans (e.g., every 120 seconds or 12 hours). It calculates the cryptographic checksum (hash) of files and compares it to a baseline. If hashes differ, an alert triggers.

# 2 Experimental Setup

The environment was configured on a Kali Linux agent registered to a Wazuh Manager. To test both modes simultaneously, two directories were created and configured in the Wazuh agent configuration file (`ossec.conf`):

- `/etc/fim-realtime-test/` – Configured with `realtime="yes"`.
- `/etc/fim-polling-test/` – Configured with `realtime="no"` (defaulting to a 120-second polling interval).

# 3 Methodology and Execution

To ensure a fair comparison, both files were modified in a single command block. We used the `tee` command to append the current timestamp to both files. This writes the literal time the command executed into the file content.

## 3.1 Step 1: Modifying the Files

The following command was executed in the Kali terminal.

```
date
echo "modified at $(date)" | sudo tee -a /etc/fim-realtime-test/
   testfile.txt
echo "modified at $(date)" | sudo tee -a /etc/fim-polling-test/
   testfile.txt
```

**Baseline Timestamp**

**Output:** The terminal returned the system date as **11:22:31 AM EST**. This is our baseline timestamp: *The exact moment the change occurred.*

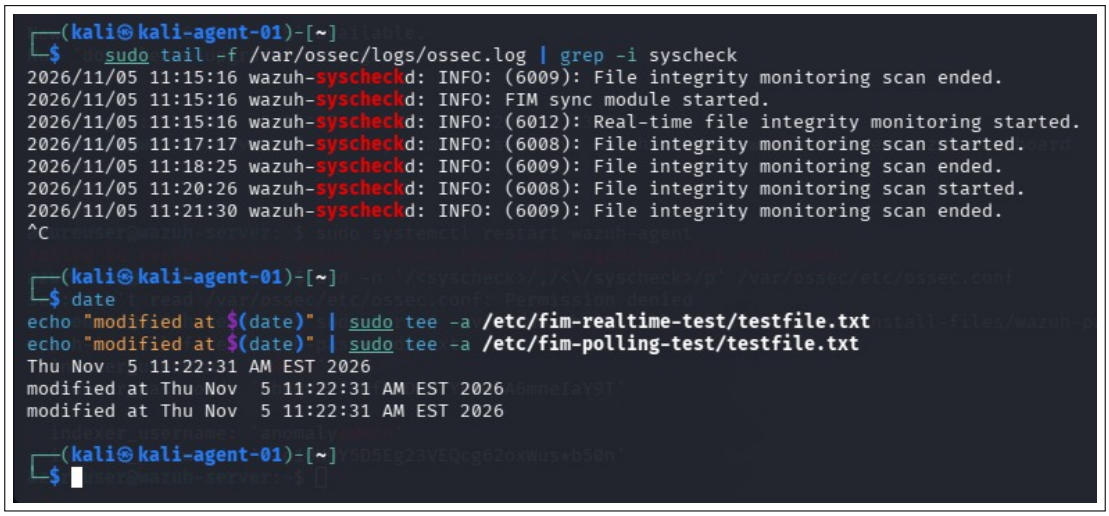



**Figure 2:** Executing the file modification commands and capturing the baseline time.

## 3.2 Step 2: Verifying the File Content

Because we embedded the timestamp into the file using `echo "modified at $(date)"`, reading the file back confirms exactly when the change happened.

```
cat /etc/fim-realtime-test/testfile.txt
cat /etc/fim-polling-test/testfile.txt
```

**Output:** Both files returned: `11:22:31`. This confirms both files were modified at the exact same second.

# 4 Results and Analysis

After the modifications, Wazuh generated alerts for both files. We queried the Wazuh Dashboard (Figure 3) filtering by `rule.groups: syscheck` to find the alert timestamps.

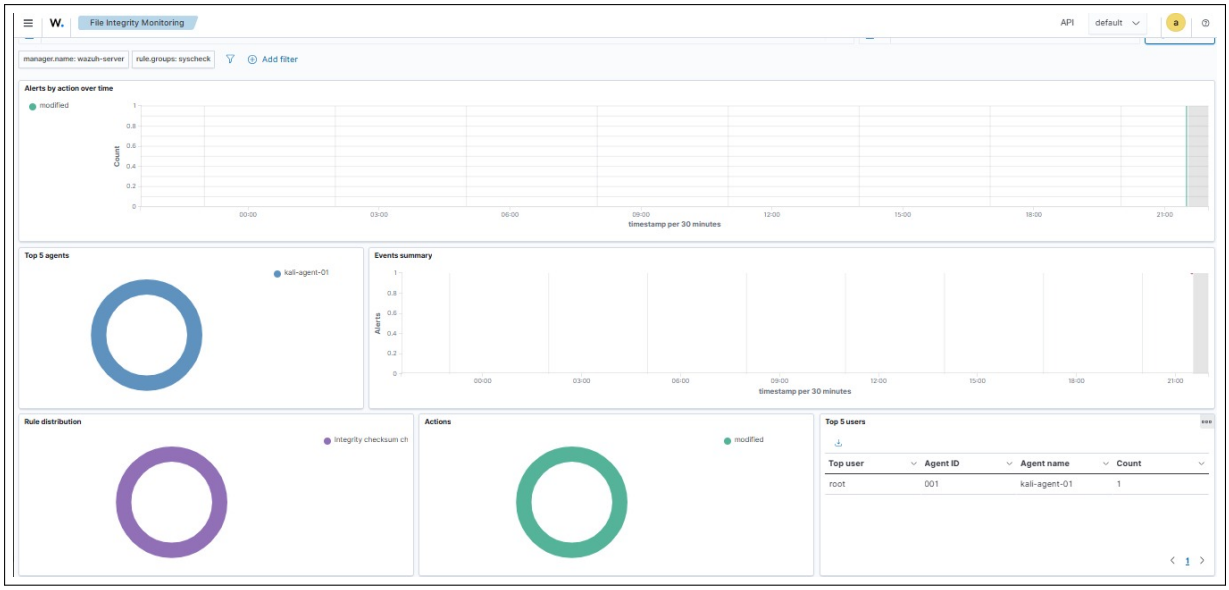



**Figure 3:** Wazuh Dashboard showing the two FIM alerts and their timestamps.

## 4.1 Alert Data

Both alerts triggered **Rule 550: Integrity checksum changed** at Level 7. The critical difference was the timestamp.

| Mode | File Path | Modification Time | Alert Timestamp |
|---|---|---|---|
| Real-Time | `/etc/fim-realtime-test/` | 11:22:31 | **21:31:54.979** |
| Polling | `/etc/fim-polling-test/` | 11:22:31 | **21:33:17.260** |

**Table 1:** Comparison of file modification times vs. alert generation times.

## 4.2 Calculating Detection Latency

To calculate the true latency, we compare the "Modification Time" against the "Alert Timestamp".

### Real-Time Latency

The real-time alert fired at **21:31:54.979**. The baseline event happened essentially at **21:31:52**.

- **Latency:** $\approx$ **2 seconds**.
- **Mechanism:** The kernel `inotify` API immediately notified the Wazuh agent when the `tee` command wrote to the file.

### Polling Latency

The polling alert fired at **21:33:17.260**.

- **Latency:** $21:33:17.260 - 21:31:54.979 =$ **82.281 seconds**.

- **Mechanism:** Because the polling interval is set to 120 seconds, the modification happened mid-cycle. The engine had to wait for the next scheduled directory scan.

## 4.3 Visualizing the Metrics

The Wazuh Dashboard (Figure 4) confirms that only the `kali-agent-01` triggered these alerts. The rule distribution confirms Rule 550 was the sole detection logic used.




**Figure 4:** Wazuh Visualizations showing Alert counts, Top Agents, and Rule distribution.

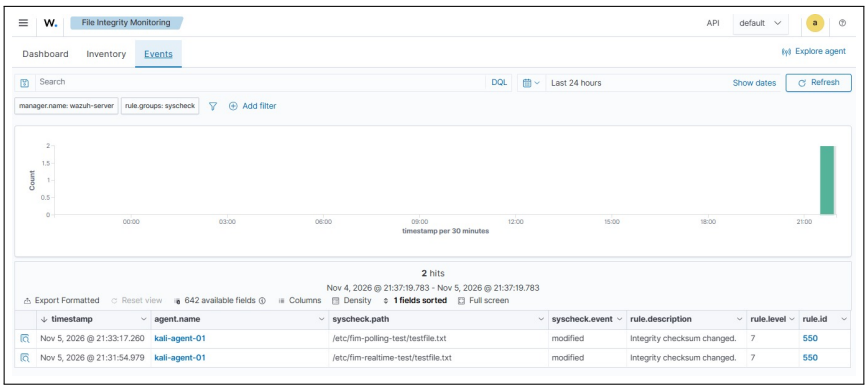



**Figure 5:** Top 5 users triggering FIM alerts (root).

# 5 Security Implications

The 82-second gap between real-time and polling detection has significant security implications:

**Real-Time vs. Polling: The Trade-off**

- **Real-Time** is essential for high-security environments (e.g., web server document roots, `/etc/passwd`). If an attacker drops a webshell, you want to know immediately.
- **Polling** is resource-efficient and should be used for less critical directories or large file systems where constant `inotify` watches would consume too much memory and CPU.

# 6 Conclusion

This lab successfully demonstrated the fundamental difference in detection latency between Real-Time and Polling FIM modes. The real-time mode detected a change in approximately 2 seconds, while the polling mode took 82 seconds. The use of the `tee` command with `$(date)` proved to be a highly effective way to establish a true baseline, as it embedded the exact execution time directly into the file, allowing us to bypass shell history limitations and calculate accurate latencies.